

Projectopdracht

Slimme voorraadweergave voor camping en appartementen

MBO niveau 4 – Softwareontwikkeling, AI-gebruik en verantwoord ontwikkelen

1. De opdracht

Een camping en appartementenverhuur verkoopt vanuit de receptie verschillende verse en gekoelde producten aan gasten, bijvoorbeeld:

- verse melk
- verse yoghurt
- ijsjes
- frisdrank
- andere kleine boodschappen

In de gemeenschappelijke ruimte willen we op een e-inkscherm kunnen laten zien welke producten op dat moment nog beschikbaar zijn.

Het scherm is alleen bedoeld om informatie te tonen. Gasten hoeven op het scherm niets te kunnen aanklikken, bestellen of betalen.

Wanneer een gast bijvoorbeeld ziet:

- ☐ Verse melk – nog 6 beschikbaar
- ☐ Yoghurt – nog 3 beschikbaar
- ☐ Ijsjes – nog 12 beschikbaar

kan de gast naar de receptie lopen om het product te kopen.

Wanneer er bij de receptie een product wordt verkocht, moet een medewerker de voorraad eenvoudig kunnen aanpassen via een mobiele webapp of app op de telefoon.

De voorraadweergave op het e-inkscherm wordt vervolgens automatisch bijgewerkt.

2. Het doel

Ontwikkel een werkend prototype waarmee medewerkers:

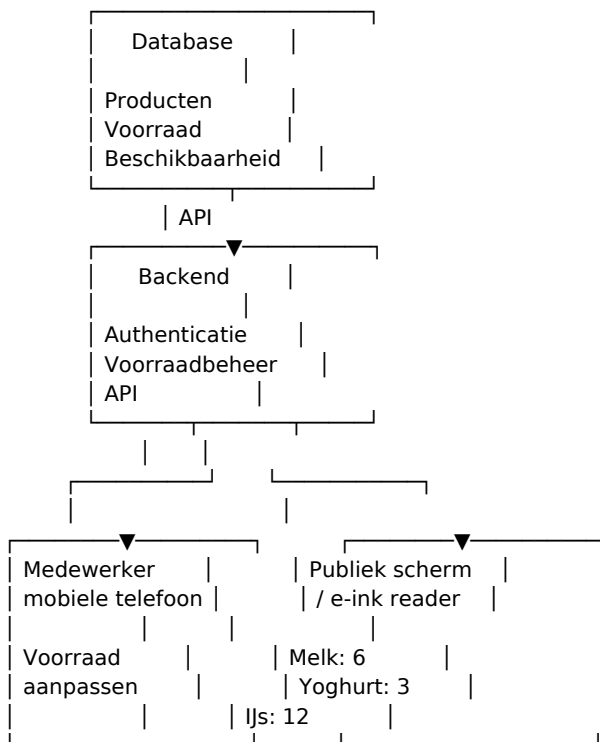
- producten kunnen aanmaken;
- de actuele voorraad kunnen bekijken;
- de voorraad snel kunnen verhogen of verlagen;
- eventueel producten tijdelijk als “niet beschikbaar” kunnen markeren.

Daarnaast ontwikkel je een publieke voorraadweergave die geschikt is om op een e-inkscherm te tonen.

De voorraadweergave wordt bijvoorbeeld iedere minuut automatisch vernieuwd.

De oplossing moet eenvoudig genoeg zijn om door twee niet-technische medewerkers te gebruiken.

3. Het systeem



Dit is slechts een voorbeeldarchitectuur. Jullie mogen zelf een andere technische oplossing kiezen, zolang jullie kunnen uitleggen waarom.

4. Functionele eisen

4.1 Voorraad bekijken

De publieke pagina toont minimaal:

- productnaam
- actuele voorraad
- duidelijke status van beschikbaarheid

Product	Voorraad
Verse melk	6
Verse yoghurt	3
Ijsjes	12

De pagina moet vooral op afstand goed leesbaar zijn. Het ontwerp moet daarom rekening houden met grote letters, duidelijke contrasten, eenvoudige vormgeving, weinig afleiding en goede leesbaarheid op een e-inkscherm.

4.2 Voorraad aanpassen

Een medewerker moet op een telefoon snel een voorraadwijziging kunnen uitvoeren.

Verse melk

Voorraad: 6

[-] 6 [+]

Opslaan

Of een andere oplossing die jullie zelf bedenken.

Het moet mogelijk zijn om bijvoorbeeld 6 → 5 te wijzigen wanneer één pak melk wordt verkocht. Het moet ook mogelijk zijn om voorraad toe te voegen wanneer nieuwe producten binnenkomen.

4.3 Meerdere medewerkers

Er zijn minimaal twee medewerkers die de voorraad moeten kunnen aanpassen.

De oplossing moet daarom rekening houden met gebruikersaccounts, inloggen en beveiligde toegang tot het voorraadbeheer. De publieke voorraadpagina hoeft niet in te loggen.

4.4 Automatisch vernieuwen

De publieke voorraadpagina moet zichzelf automatisch vernieuwen.

Iedere 60 seconden:
pagina opnieuw laden

Wanneer een medewerker bij de receptie de voorraad verandert, moet de wijziging dus uiterlijk ongeveer één minuut later zichtbaar zijn op het scherm.

5. Wat hoeft NIET?

Om de opdracht haalbaar te houden, hoeft het systeem in de eerste versie niet:

- producten online te laten bestellen;
- betalingen te verwerken;
- een webshop te zijn;
- een winkelmandje te hebben;
- klantaccounts te hebben;
- voorraad op de telefoon van gasten beschikbaar te maken;
- pushberichten te versturen;
- een uitgebreide kassakoppeling te hebben.

Het doel is: “Laat op één scherm zien wat er nog beschikbaar is en geef medewerkers een snelle manier om de voorraad bij te werken.”

6. Technische vrijheid

Jullie mogen zelf bepalen welke technologieën jullie gebruiken.

- Front-end: HTML/CSS/JavaScript, React, Vue, Svelte of een andere geschikte technologie.
- Backend: Node.js, Python, PHP, C# of een andere geschikte technologie.
- Database: SQLite, PostgreSQL, MySQL/MariaDB, Supabase, Firebase of een andere geschikte oplossing.
- Hosting: eigen server, VPS, cloudplatform, platform-as-a-service of een andere geschikte oplossing.

Let op: een technologie kiezen omdat AI zegt dat het “de beste” is, is geen onderbouwing. Jullie moeten kunnen uitleggen: Waarom hebben wij voor deze oplossing gekozen?

7. Minimale technische eisen

Publieke pagina

- actuele voorraad tonen
- automatisch vernieuwen
- geschikt zijn voor een groot scherm
- geen mogelijkheid bieden om voorraad te wijzigen

Beheergedeelte

- beveiligd met login
- voorraad kunnen verhogen
- voorraad kunnen verlagen
- voorraad kunnen instellen
- meerdere producten kunnen beheren

Backend

- API voor het ophalen van voorraad
- API voor het wijzigen van voorraad
- correcte verwerking van fouten
- invoervalidatie

Database

De voorraadgegevens moeten persistent worden opgeslagen. Wanneer de server opnieuw wordt gestart, moet de voorraad dus niet verdwenen zijn.

8. Denk na over fouten

Een professioneel systeem houdt rekening met situaties die mis kunnen gaan.

Onderzoek bijvoorbeeld wat gebeurt wanneer:

- twee medewerkers tegelijkertijd een product verkopen;
- iemand de voorraad op -1 probeert te zetten;
- de internetverbinding tijdelijk wegvalt;
- de database niet bereikbaar is;
- een medewerker een verkeerd getal invoert;
- een onbekende gebruiker probeert de voorraad te wijzigen;
- het e-inkscherm geen verbinding heeft;
- de publieke pagina wordt geopend terwijl de server offline is.

Jullie hoeven niet iedere situatie volledig op te lossen, maar jullie moeten wel aantonen dat jullie erover hebben nagedacht.

9. Security

Omdat de voorraad kan worden aangepast, mag iedereen niet zomaar toegang krijgen tot het beheergedeelte.

Onderzoek minimaal:

- hoe authenticatie werkt;
- hoe wachtwoorden veilig worden opgeslagen;
- hoe een medewerker wordt ingelogd;
- hoe wordt voorkomen dat iemand zonder toestemming voorraad wijzigt;
- welke API-endpoints publiek toegankelijk zijn;
- welke API-endpoints beveiligd moeten zijn.

Belangrijk uitgangspunt: De publieke pagina mag voorraad lezen, maar mag de voorraad niet wijzigen.

10. UX-opdracht

Ontwerp twee verschillende gebruikerservaringen.

Gebruiker A - de gast

De gast staat in de gemeenschappelijke ruimte en kijkt van enkele meters afstand naar het scherm.

De gast moet binnen enkele seconden kunnen begrijpen welke producten beschikbaar zijn, hoeveel er nog zijn en welke producten uitverkocht zijn.

Gebruiker B - de medewerker

De medewerker staat achter de receptie met een mobiele telefoon. Een voorraadowijziging moet zo snel mogelijk kunnen worden uitgevoerd.

Denk bijvoorbeeld aan: Gast koopt 1 melk → medewerker opent app → drukt op min → klaar.

Maak eerst een eenvoudige wireframe voordat jullie de interface bouwen.

11. Extra uitdaging: e-ink

Onderzoek wat een e-inkscherm anders maakt dan een normaal LCD- of OLED-scherm.

- stroomverbruik
- refresh rate
- kleuren
- ghosting
- leesbaarheid
- browserondersteuning
- wifi
- schermresolutie
- automatische refresh
- permanent ingeschakeld laten staan

Onderzoek vervolgens of een e-reader geschikt is voor deze toepassing.

Een e-reader is namelijk niet automatisch hetzelfde als een klein e-inkdisplay dat bedoeld is om een webpagina permanent weer te geven. Jullie mogen daarom ook een andere technische oplossing voorstellen.

12. AI is onderdeel van de opdracht

Tijdens dit project mogen jullie AI gebruiken. Sterker nog: AI-gebruik is onderdeel van de beoordeling.

Maar AI is geen vervanging voor jullie eigen technische inzicht. Jullie zijn verantwoordelijk voor het eindresultaat.

Als AI zegt: “Gebruik technologie X, want dat is de beste oplossing.” dan is dat niet automatisch waar.

Jullie moeten onderzoeken: klopt dit? Waarom klopt het? Wanneer klopt het niet? Welke alternatieven zijn er? Wat zijn de nadelen? Hebben we het zelf getest?

13. AI-logboek

Iedere student houdt gedurende het project een persoonlijk AI-logboek bij.

Voor iedere relevante AI-interactie leg je vast:

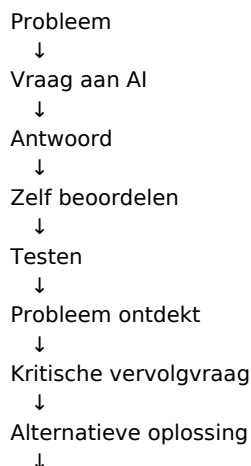
- datum;
- welke AI-tool is gebruikt;
- de volledige prompt/vraag;
- het antwoord van de AI;
- wat je met het antwoord hebt gedaan;
- of je het antwoord hebt gecontroleerd;
- welke vervolgvraag je hebt gesteld;
- waarom je die vervolgvraag hebt gesteld;
- wat uiteindelijk in het project is gebruikt.

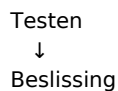
Datum	Prompt	Antwoord AI	Wat heb ik ermee gedaan?	Controle
12-09	Hoe kan ik automatisch een webpagina genereren?	AI stelt oplossing X voor	Uitgetest	Werkt
12-09	Waarom is X geschikt voor e-ink?	AI geeft uitleg	Niet volledig overtuigd	Andere bronnen geraadpleegd
13-09	Wat zijn nadelen van X op e-ink?	AI noemt ...	Oplossing aangepast	Getest

14. Kritisch AI-gebruik

Het is nadrukkelijk niet de bedoeling om alleen een enorme prompt aan AI te geven: “Maak deze hele applicatie voor mij.” en vervolgens de code blind te kopiëren.

Jullie moeten laten zien dat jullie AI als ontwikkelpartner gebruiken.





De kwaliteit van de vragen die jullie aan AI stellen, is onderdeel van het leerproces.

15. Voorbeeld van kritisch AI-gebruik

Stel dat AI voorstelt: “Sla de voorraad op in de browser van de gebruiker.”

Dan moeten jullie jezelf afvragen: Is dat logisch?

Waarschijnlijk niet. De voorraad moet namelijk door meerdere telefoons én het e-inkscherm gedeeld worden.

Een goede vervolgvraag kan zijn:

“De voorraad moet door meerdere apparaten gedeeld worden. Is lokale browseropslag hiervoor geschikt? Leg uit waarom wel of niet en geef alternatieven.”

Daarna moeten jullie het antwoord testen en onderbouwen.

AI mag helpen, maar jullie blijven verantwoordelijk voor de technische keuzes.

16. AI mag helpen met

- uitleg van programmeerconcepten;
- brainstormen;
- architectuurvoorstellen;
- databaseontwerp;
- API-ontwerp;
- codevoorbeelden;
- debugging;
- foutmeldingen verklaren;
- testgevallen bedenken;
- documentatie verbeteren;
- code review;
- security-vraagstukken;
- UX-ideeën.

Maar: jullie moeten de code en oplossingen begrijpen die jullie inleveren.

De docent mag daarom vragen: “Waarom heb je dit zo gebouwd?”, “Wat gebeurt er hier?” of “Kun je deze code zonder AI aanpassen?”

17. Testen

Ontwikkel minimaal 15 testgevallen.

Test	Verwacht resultaat
Voorraad melk = 6	6 wordt getoond
Eén melk verkocht	Voorraad wordt 5

Test	Verwacht resultaat
Voorraad = 0	Product wordt als uitverkocht weergegeven
Voorraad -1 invoeren	Wordt geweigerd
Niet-ingelogde gebruiker probeert voorraad te wijzigen	Toegang geweigerd
Pagina wordt vernieuwd	Nieuwe voorraad zichtbaar
Database wordt herstart	Voorraad blijft bestaan

Voeg zelf minimaal tien relevante tests toe.

18. Documentatie

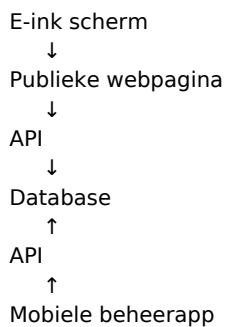
Lever naast de software een technische documentatie op.

Probleemanalyse

- Wat is het probleem?
- Voor wie lossen jullie het op?
- Welke eisen zijn er?

Architectuur

Leg uit hoe de onderdelen samenwerken.



Technologiekeuzes

Leg uit welke programmeertalen, frameworks, database en hosting jullie gebruiken en waarom jullie deze keuzes hebben gemaakt.

Datamodel

Laat zien hoe de gegevens worden opgeslagen.

```

PRODUCT
-----
id
naam
voorraad
prijs
actief
  
```

Jullie mogen uiteraard een ander datamodel ontwerpen.

Security

Beschrijf authenticatie, autorisatie, wachtwoordbeveiliging, API-beveiliging en mogelijke risico's.

Testplan

Beschrijf jullie testgevallen en resultaten.

AI-verantwoording

Beschrijf welke AI-tools zijn gebruikt, waarvoor AI is gebruikt, welke belangrijke adviezen AI gaf, welke AI-antwoorden fout of onvolledig waren, hoe jullie dit hebben ontdekt en welke keuzes jullie uiteindelijk zelf hebben gemaakt.

19. Eindpresentatie

Presenteer jullie oplossing aan de klas. De presentatie duurt ongeveer 10 minuten.

Laat minimaal zien:

- het probleem;
- jullie oplossing;
- de publieke voorraadweergave;
- de mobiele beheeromgeving;
- een voorraadwijziging;
- de wijziging die vervolgens op het scherm verschijnt;
- de technische architectuur;
- een voorbeeld van een AI-antwoord dat jullie kritisch hebben beoordeeld;
- wat jullie daarvan hebben geleerd.

Daarna volgt een korte demonstratie.

20. Beoordeling

Onderdeel	Gewicht
Werkende applicatie	25%
Functionaliteit & gebruiksgemak	15%
Technische kwaliteit	15%
Security & foutafhandeling	10%
Testen	10%
Documentatie	10%
AI-logboek & kritisch AI-gebruik	10%
Presentatie	5%
Totaal	100%

Een applicatie die technisch perfect werkt, maar waarbij studenten niet kunnen uitleggen hoe of waarom, is geen volledige voldoende.

21. Eindproduct

Aan het einde leveren jullie minimaal op:

Software

- publieke voorraadpagina

- mobiele beheeromgeving
- backend/API
- database
- authenticatie

Documentatie

- probleemanalyse
- architectuur
- technische keuzes
- datamodel
- securityanalyse
- testplan
- testresultaten
- AI-verantwoording

Persoonlijk

- persoonlijk AI-logboek

Presentatie

- demonstratie van het werkende systeem

22. Bonusopdrachten

Bonus 1 - Voorraadmutaties

Houd niet alleen de actuele voorraad bij, maar ook:

13:42 - melk - 1 verkocht - medewerker Jan
 13:51 - melk - 6 → 12 - voorraad aangevuld
 14:03 - ijsje - 1 verkocht - medewerker Marieke

Bonus 2 - Automatische status

Toon bijvoorbeeld: ☐ Ruim op voorraad, ☐ Bijna op, ☐ Uitverkocht. Onderzoek zelf welke grenzen logisch zijn.

Bonus 3 - Meerdere locaties

Ondersteun meerdere verkooppunten of gebouwen.

Bonus 4 - Offline gedrag

Onderzoek wat er gebeurt wanneer de mobiele telefoon tijdelijk geen internet heeft.

Bonus 5 - Echte e-ink hardware

Laat de applicatie daadwerkelijk draaien op een e-inkapparaat en onderzoek de beperkingen daarvan.

Bonus 6 - Wijzigingsgeschiedenis

Maak inzichtelijk wie welke voorraadwijziging heeft uitgevoerd.

23. De belangrijkste vraag

Jullie bouwen niet zomaar “een website”.

Jullie bouwen een klein informatiesysteem voor een echte praktijksituatie.

Daarbij moeten jullie steeds kunnen uitleggen:

Waarom hebben we dit zo opgelost?

En wanneer AI een oplossing aandraagt:

Hoe weten we dat AI gelijk heeft?

Dat is uiteindelijk belangrijker dan de hoeveelheid code die jullie schrijven.